

Graduate Machine Learning: Problem Set 1

Newton's Method, Gradient Descent, and Linear Regression
Optimization
Spring 2026

Instructor: [Your Name]
Madras School of Economics
MSE/IISc/Stanford

March 18, 2026

Instructions:

- **Problem 1 is mandatory.** It covers core optimization concepts essential for machine learning.
- **Submission:** Submit a typeset PDF report (LaTeX preferred) with both written answers and code.
- **Code:** All code should be well-commented, reproducible, and submitted separately as .py files.
- **Figures:** Include clear plots with labels, legends, and captions.
- **Deadline:** Due [DATE] at 11:59 PM via [SUBMISSION PLATFORM].
- **Collaboration Policy:** You may discuss ideas with classmates, but write solutions independently. Cite all sources.

Contents

Problem 1: Newton's Method vs. Gradient Descent	3
---	---

Problem 1: Newton's Method vs. Gradient Descent for Linear Regression

Background and Motivation

In this problem, we study **optimization algorithms** for fitting linear regression models. We focus on three distinct approaches:

1. **Explicit matrix inversion** (normal equations)
2. **Gradient descent** (first-order method)
3. **Newton's method** (second-order method)

Understanding these methods is critical because:

- **Matrix inversion** is theoretically elegant but computationally expensive for large d (number of features).
- **Gradient descent** scales well to large problems but converges slowly.
- **Newton's method** converges quadratically (very fast near optimum) but requires computing a Hessian matrix.

For **sparse matrices** (mostly zeros), specialized algorithms can exploit sparsity to achieve significant speedups. This problem compares all three approaches on increasingly large regression problems with varying sparsity levels.

Part 1: Theoretical Background

1A. Problem Setup: Least Squares Regression

We consider the unconstrained optimization problem:

$$(1) \quad \min_{\mathbf{w} \in \mathbb{R}^d} f(\mathbf{w}) := \frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{2n} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

where:

- $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the **design matrix** (n samples, d features)
- $\mathbf{y} \in \mathbb{R}^n$ is the **response vector**
- $\mathbf{w} \in \mathbb{R}^d$ is the **weight vector** we optimize
- We assume $\mathbf{X}$ has full rank ($\text{rank } d \leq n$)

Key observation: This is a **convex quadratic** function. The loss surface is a paraboloid in weight space.

Exercise 1A.1: Derive the Gradient Compute the gradient of $f(\mathbf{w})$ with respect to $\mathbf{w}$:

$$(2) \quad \nabla f(\mathbf{w}) = \frac{\partial f}{\partial \mathbf{w}}$$

Hint: Expand $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ and use the product rule and chain rule for matrix calculus. Recall that $\frac{\partial}{\partial \mathbf{w}}(\mathbf{X}\mathbf{w}) = \mathbf{X}^T$ and $\frac{\partial}{\partial \mathbf{w}}(\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}) = 2\mathbf{X}^T \mathbf{X} \mathbf{w}$.

Exercise 1A.2: Derive the Hessian Compute the Hessian (matrix of second derivatives) of $f(\mathbf{w})$:

$$(3) \quad \mathbf{H} = \nabla^2 f(\mathbf{w}) = \frac{\partial^2 f}{\partial \mathbf{w} \partial \mathbf{w}^T}$$

What is the dimension of $\mathbf{H}$? Is it symmetric? Is it positive definite? (Justify your answers.)

Exercise 1A.3: Optimal Solution Show that the minimum of $f(\mathbf{w})$ is achieved at:

$$(4) \quad \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

This is called the **normal equations** or **ordinary least squares (OLS) solution**.
Hint: Set $\nabla f(\mathbf{w}) = 0$ and solve for $\mathbf{w}$.

1B. Gradient Descent Algorithm

Gradient descent is a **first-order iterative method**. Starting from an initial guess $\mathbf{w}^{(0)}$, it updates:

$$(5) \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha_t \nabla f(\mathbf{w}^{(t)})$$

where:

- $\alpha_t > 0$ is the **step size** (learning rate) at iteration t
- $\nabla f(\mathbf{w}^{(t)})$ is the gradient (direction of steepest ascent)
- The update moves in the opposite direction (steepest descent)

Exercise 1B.1: Gradient Descent for Least Squares For the least squares problem (1), write out the gradient descent update rule explicitly:

$$(6) \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha_t \nabla f(\mathbf{w}^{(t)})$$

Substitute the gradient you derived in Exercise 1A.1.

Exercise 1B.2: Convergence Rate Gradient descent converges to the optimum, but how fast? For the quadratic loss, the convergence rate depends on the **condition number** of $\mathbf{X}^T \mathbf{X}$.

Definition: The condition number is:

$$(7) \quad \kappa = \frac{\lambda_{\max}(\mathbf{X}^T \mathbf{X})}{\lambda_{\min}(\mathbf{X}^T \mathbf{X})}$$

where $\lambda_{\max}$ and $\lambda_{\min}$ are the largest and smallest eigenvalues.

Fact (from optimization theory): With step size $\alpha_t = \alpha$ (constant), gradient descent for strongly convex quadratic functions converges as:

$$(8) \quad \|\mathbf{w}^{(t)} - \mathbf{w}^*\|_2^2 \leq C \rho^t \|\mathbf{w}^{(0)} - \mathbf{w}^*\|_2^2$$

where the convergence rate is $\rho = \frac{\kappa-1}{\kappa+1}$ (approximately, for well-tuned α).

Question: If $\kappa = 1$ (perfectly conditioned), what is ρ ? If $\kappa = 100$ (ill-conditioned), what is ρ ? How many iterations are needed to reduce error by a factor of 100 in each case?

1C. Newton's Method Algorithm

Newton's method is a **second-order iterative method**. It uses both the gradient **and** the Hessian:

$$(9) \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{H}^{-1} \nabla f(\mathbf{w}^{(t)})$$

where:

- $\mathbf{H} = \nabla^2 f(\mathbf{w}^{(t)})$ is the Hessian at iteration t
- We subtract the “Newton direction” $\mathbf{H}^{-1} \nabla f(\mathbf{w}^{(t)})$

Geometric Intuition In one dimension, Newton's method approximates the function by a quadratic Taylor expansion and jumps directly to the minimum of the parabola:

$$(10) \quad f(w) \approx f(w^{(t)}) + f'(w^{(t)})(w - w^{(t)}) + \frac{1}{2} f''(w^{(t)})(w - w^{(t)})^2$$

Setting the derivative to zero and solving gives the Newton update.

In multiple dimensions, the intuition is similar: we use a quadratic approximation (via the Hessian) to the loss surface.

Exercise 1C.1: Newton's Method for Least Squares For the least squares problem, the Hessian is constant: $\mathbf{H} = \frac{1}{n} \mathbf{X}^T \mathbf{X}$ (from Exercise 1A.2).

Write out the Newton's method update:

$$(11) \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \left(\frac{1}{n} \mathbf{X}^T \mathbf{X} \right)^{-1} \nabla f(\mathbf{w}^{(t)})$$

Substitute the gradient. Simplify to show that Newton's method reaches the optimum in **one step**!

Exercise 1C.2: Why Newton's Method Converges in One Step Explain why Newton's method converges to $\mathbf{w}^*$ in a single iteration for a quadratic loss function.

Hint: The function is exactly quadratic, so the quadratic Taylor approximation is exact. The minimum of the true function equals the minimum of the approximation.

Exercise 1C.3: Quadratic Convergence More generally, for non-quadratic functions, Newton’s method exhibits **quadratic convergence**:

$$(12) \quad \|\mathbf{w}^{(t+1)} - \mathbf{w}^*\|_2 \leq C \|\mathbf{w}^{(t)} - \mathbf{w}^*\|_2^2$$

In words: the error squares at each iteration (once close enough to optimum). This is much faster than gradient descent’s linear convergence.

Question: If the error is 10^{-1} at iteration t :

- What is the error after 1 Newton step (assuming constant $C = 1$)?
- After 2 Newton steps?
- How many Newton steps to reach error 10^{-10} ?

Compare with gradient descent (using $\rho = 0.98$ for a moderately conditioned problem). How many gradient descent steps are needed?

Part 2: Implementation and Comparison

In this part, you will implement all three methods and compare them on large sparse linear regression problems.

2A. Algorithm Implementations

Exercise 2A.1: Implement Three Methods Implement the following in Python (numpy/scipy):

1. **Normal Equations:** Directly solve $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
For sparse matrices, use `scipy.sparse.linalg.spsolve` instead of explicit inversion.
2. **Gradient Descent:** Iterate using (5) with constant step size $\alpha = \frac{1}{\lambda_{\max}(\mathbf{X}^T \mathbf{X})}$ (or use line search).
For sparse matrices, use matrix-vector products (which are fast for sparse $\mathbf{X}$).
3. **Newton’s Method:** Iterate using (9), solving a linear system at each step.
Use `scipy.sparse.linalg.spsolve` for efficiency.

Requirements:

- Each function should accept $\mathbf{X}$, $\mathbf{y}$, and algorithm-specific parameters (e.g., step size α , max iterations, tolerance).

- Return the final weight vector $\mathbf{w}$ and a history of losses at each iteration.
- Use convergence criterion: stop when $\|\nabla f(\mathbf{w})\|_2 < \text{tol}$ or max iterations reached.

Exercise 2A.2: Example Synthetic Data Generate synthetic regression data as follows:

```

1 import numpy as np
2 from scipy.sparse import random as sp_random
3 from scipy.sparse import csr_matrix
4
5 np.random.seed(42)
6
7 # Parameters
8 n_samples = 10000      # Number of training samples
9 n_features = 5000      # Number of features
10 sparsity = 0.99        # 99% zeros in design matrix
11
12 # Generate sparse design matrix
13 X = sp_random(n_samples, n_features, density=1-sparsity,
14              format='csr', random_state=42)
15
16 # Generate true weights (sparse)
17 w_true = np.random.randn(n_features)
18 w_true[np.random.choice(n_features, int(0.95*n_features),
19                          replace=False)] = 0
20
21 # Generate responses with noise
22 y = X.dot(w_true) + np.random.randn(n_samples) * 0.1
23
24 # Normalize (important for numerical stability)
25 X_mean = np.array(X.mean(axis=0)).flatten()
26 X_std = np.array(np.sqrt(X.power(2).mean(axis=0))).flatten() + 1e-8
27 X = (X - X_mean) / X_std
28
29 y_mean = y.mean()
30 y = y - y_mean

```

Adjust n_{samples} and n_{features} as needed, and try different sparsity levels.

2B. Runtime Comparison

Exercise 2B.1: Wall-Clock Time Measurement For each method, measure the actual wall-clock time:


```

1 import time
2
3 def time_method(method_name, X, y, **kwargs):
4     """Time a single optimization method."""
5     start = time.time()
6     w_opt, loss_history = method(X, y, **kwargs)
7     elapsed = time.time() - start
8     return w_opt, loss_history, elapsed
9
10 # Test parameters
11 sizes = [(1000, 500), (5000, 2000), (10000, 5000)]
12 sparsities = [0.90, 0.95, 0.99]
13
14 results = {}
15 for n, d in sizes:
16     for s in sparsities:
17         # Generate data
18         X = generate_sparse_data(n, d, s)
19         y = generate_response(X)
20
21         # Time each method
22         w_ne, loss_ne, t_ne = time_method('normal_eqs', X, y)
23         w_gd, loss_gd, t_gd = time_method('gradient_descent', X, y,
24                                           max_iter=1000, tol=1e-6)
25         w_nm, loss_nm, t_nm = time_method('newtons_method', X, y,
26                                           max_iter=100, tol=1e-6)
27
28         results[(n, d, s)] = {
29             'normal_eqs': t_ne,
30             'gradient_descent': t_gd,
31             'newtons_method': t_nm
32         }

```

Report: For each problem size and sparsity level, create a table showing:

- Wall-clock time for each method
- Number of iterations to convergence (for iterative methods)
- Final loss achieved
- Whether solutions are consistent (same $\mathbf{w}$?)

Exercise 2B.2: Visualization Create the following visualizations:

1. **Runtime vs. Problem Dimension:** Plot wall-clock time on the y -axis and number of features d on the x -axis. Show three curves (one for each method). Use log-log scale.
2. **Loss Convergence Curves:** For a fixed problem size (e.g., $n = 5000, d = 2000$), plot loss $f(\mathbf{w}^{(t)})$ vs. iteration number t for gradient descent and Newton's method. Use log scale for loss.
3. **Runtime vs. Sparsity:** Fix problem size (e.g., $n = 10000, d = 5000$) and vary sparsity from 0.50 to 0.99. Plot time vs. sparsity for each method.

2C. Theoretical Analysis

Exercise 2C.1: Complexity Analysis For each method, derive the computational complexity (number of floating-point operations, FLOPs):

- **Normal Equations:** Computing $\mathbf{X}^T\mathbf{X}$ costs $O(nd^2)$ (for dense $\mathbf{X}$) or $O(\text{nnz} \cdot d)$ for sparse $\mathbf{X}$ (where nnz is the number of non-zeros). Solving the linear system $(\mathbf{X}^T\mathbf{X})\mathbf{w} = \mathbf{X}^T\mathbf{y}$ costs $O(d^3)$.
For sparse matrices, use Cholesky decomposition: $O(d^3)$ (worst case, but often much faster due to sparsity).
- **Gradient Descent:** One iteration costs $O(\text{nnz})$ (single matrix-vector product $\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$). With T_{gd} iterations, total cost is $O(T_{\text{gd}} \cdot \text{nnz})$.
- **Newton's Method:** One iteration costs $O(\text{nnz})$ (compute gradient) plus $O(d^3)$ (solve linear system). With T_{nm} iterations, total cost is $O((T_{\text{nm}} \cdot \text{nnz}) + (T_{\text{nm}} \cdot d^3))$.

Question: For sparse $\mathbf{X}$ with $\text{nnz} = O(nd)$ (proportional to matrix size):

- When is Newton's method faster than gradient descent? (In terms of d , sparsity, and convergence rates.)
- When is normal equations fastest?
- Create a complexity comparison table for $n = 10000, d \in \{1000, 5000, 10000\}$.

Exercise 2C.2: Theoretical Validation Estimate the expected runtimes based on theory and compare with empirical results.

1. For gradient descent with $\kappa = 100$ and tolerance $\text{tol} = 10^{-6}$:
 - How many iterations are needed? (Use the convergence rate $\rho = \frac{\kappa-1}{\kappa+1}$ and require $\rho^T < \text{tol}^2$.)
 - What is the predicted runtime? (iterations $\times$ cost per iteration)
2. For Newton's method with quadratic convergence:
 - How many iterations to reach tolerance 10^{-6} (starting error 10^{-1})?
 - What is the predicted runtime?
3. Compare predictions with empirical timings from Exercise 2B.1.

Exercise 2C.3: Condition Number Effects The condition number κ dramatically affects gradient descent convergence. Investigate this:

1. Generate a design matrix $\mathbf{X}$ with known condition number (use SVD decomposition: create $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ where $\mathbf{S}$ is diagonal with controlled singular values).
2. For several values of κ (e.g., 1, 10, 100, 1000), run gradient descent and Newton's method.
3. Plot: number of iterations to convergence vs. κ (log-log scale).
4. Compare: Newton's method should be unaffected by κ (needs 1 iteration for quadratic!), while gradient descent should scale roughly as $\log(\kappa)$ iterations.

Part 3: Discussion and Insights

Exercise 3.1: Summary Table Create a comprehensive summary table comparing the three methods:

Aspect	Normal Eqs.	Gradient Descent	Newton's Method
Time per iteration	$O(d^3)$	$O(\text{nnz})$	$O(\text{nnz} + d^3)$
Number of iterations	1	$O(\log \kappa)$	1 (quadratic)
Total time	$O(d^3)$	$O(\text{nnz} \log \kappa)$	$O(\text{nnz} + d^3)$
Memory	$O(d^2)$	$O(\text{nnz})$	$O(\text{nnz} + d^2)$
Robust to ill-conditioning?	No	No	No
Parallelizable?	Hard	Yes	Hard

Exercise 3.2: When to Use Each Method? Based on your experiments, write a brief (200–300 words) recommendation guide:

- When should practitioners use normal equations?
- When should they use gradient descent?
- When should they use Newton's method?
- How does sparsity change these recommendations?

Exercise 3.3: Real-World Implications

1. Gradient descent is the workhorse of deep learning. Why is Newton's method not used for neural networks?

Hint: Consider: What is the Hessian of a deep neural network? Is it sparse? Can we compute $\mathbf{H}^{-1}$ efficiently?

2. Normal equations require storing $\mathbf{X}^T \mathbf{X}$ (dimension $d \times d$). For $d = 1,000,000$, how much memory is needed? Is this practical?
3. How would you modify gradient descent for a problem that doesn't fit in memory? (Hint: stochastic gradient descent, mini-batch processing.)

Submission Guidelines

1. **Written Report:** Write your answers to all exercises in a single PDF (typeset, not handwritten).
2. **Code:** Submit clean, well-commented Python code. Organize into functions:

```
1  def normal_equations(X, y, **kwargs):
2      """Solve via normal equations."""
3      ...
4      return w_opt, loss_history
5
6  def gradient_descent(X, y, learning_rate=0.01, max_iter=1000,
7                      tol=1e-6):
8      """Gradient descent for least squares."""
9      ...
10     return w_opt, loss_history
11
12 def newtons_method(X, y, max_iter=100, tol=1e-6):
13     """Newton's method for least squares."""
14     ...
15     return w_opt, loss_history
```

3. **Figures:** All plots should have:
 - Clear title and axis labels
 - Legend (if multiple curves)
 - Caption explaining what is shown
 - Proper scale (log/linear) chosen appropriately
4. **Reproducibility:** Your code should be runnable from scratch. Set random seeds for reproducibility.
5. **Clarity:** Write clearly. Explain your reasoning, not just formulas. If code is complex, add comments.

Appendix: Matrix Calculus Cheat Sheet

For reference:

$$(13) \quad \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}) = 2\mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$(14) \quad \frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{X} \mathbf{w}) = \mathbf{X}^T \mathbf{y}$$

$$(15) \quad \frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^T \mathbf{X}^T \mathbf{y}) = \mathbf{X}^T \mathbf{y}$$

$$(16) \quad \frac{\partial}{\partial \mathbf{w}} \|\mathbf{X} \mathbf{w} - \mathbf{y}\|_2^2 = 2\mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$$

References

- Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. (Chapter 9-11 on algorithms)
- Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer. (Chapters 2-3 on gradient descent and Newton's method)
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. (Chapter 3 on linear regression, Appendix on optimization)
- Stanford CS229 Lecture Notes: <https://cs229.stanford.edu/> (Linear Regression, Optimization notes)
- Berkeley CS189 Course Materials: <https://people.eecs.berkeley.edu/~jrs/189/> (Regression lectures and homework)